Apply the Vocabulary of the Architecture

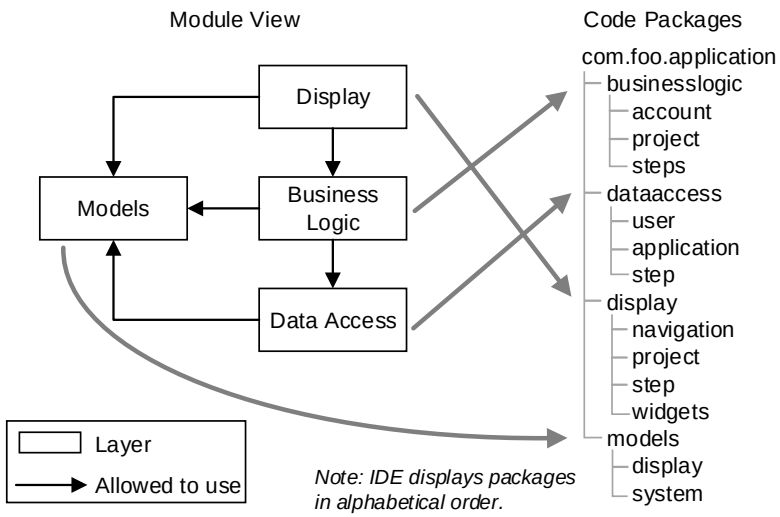
Terminology mismatch is a common source of confusion when moving from architectural abstractions to code. The architecture talks about *layers*, *services*, and *filters*, but the code implements *packages*, *classes*, and *methods*. The simplest way to embed a model is to use the vocabulary of the architecture.

If we're using layers, then let's call our code packages layers. If we've adopted a pipe-and-filter pattern, our classes should be named *pipes* and *filters*. If we talk about *pilots* and *navigators* in our system metaphors, then we should use these words as names for types and instances.

Embedding the domain model into the code is another way to shrink the gap between models and code. Modeling code after domain concepts is a common practice in object-oriented programming. Many frameworks, including object-relational mappers and actor-based systems, assume (or at least strongly encourage) a domain model as part of the implementation. Modeling the domain in this way is a core tenet of domain-driven design and several other design methodologies. Similarly, event-based and reactive patterns lean heavily on insights derived from event models derived from domain workflows.

Organize Code to Make Patterns Obvious

Good naming is just the beginning. How we organize the code dramatically affects architectural structures in code. A compiler will happily build your Java application whether every class is in the same file or classes are logically organized around thematic packages. The following example shows how to organize layers into their own code packages:



There are other ways to organize this code. Instead of using traditional layers, we could have created functionality-oriented modules. With this pattern, all classes required to complete a functional area are contained within a single package. Classes external to the functional package would not be able to access the business logic or data access classes.

Organizing code so that it matches the designed module structures should be a standard best practice. Patterns on a whiteboard don't promote quality attributes. The code does. If you can't see the pattern in the built system, then it doesn't exist. If the pattern wasn't implemented, then desired quality attributes can't be satisfied as designed. Simon Brown has done a lot of work in this area and shares several examples in *Software Architecture for Developers [Bro16]*.

Organizing code into packages that correspond to architectural elements is the least we can do. Even better is to enforce the relations so that it becomes virtually impossible to violate the architecture.

Enforce Relations Among Elements

The problem with most architectures is that they rely on discipline and vigilance to maintain conceptual integrity. Instead of relying on discipline alone, look for ways to enforce the architecture in code. It is impossible (or at least *really* difficult) to disobey design decisions enforced by the code.

The degree to which we can enforce the architecture depends on the type of structures we're dealing with, the programming languages, operating environment, and the other technical factors.

Module Structures

Module structures are the simplest to see in the code but often the most difficult to enforce. In most modern programming languages, we can enforce an *allowed to use* relation by limiting access to specific modules. If that fails, it's usually possible to distribute modules as a library with decent documentation.

When we can't enforce relations, we can at least monitor them. Use static analysis tools to identify violations of the *uses*, *allowed to use*, or *requires* relations. In some programming languages, you might use types creatively to render relations among elements visible and easy to monitor.

Component and Connector Structures

One approach to enforcing component and connector models is to design the system to fail fast when the architecture is violated. Design by contract, first defined in *Object-Oriented Software Construction [Mey97]*, is an approach where pre-conditions, post-conditions, and invariants are added to the code

and checked at runtime. When a developer violates the contract, the application throws an error and execution ceases. Contracts work at many granularities of abstraction, including objects, services, and processes across threads.

Another common approach to enforcing C&C models is to prevent connections between components that should not be connected. One example is to require authentication between components, a common practice when connecting to a data source from a data access tier.

The swift rise in popularity of microservices architecture is in part due to the fact that the pattern makes domain models visible and enforceable at runtime. Enforcing *allowed to use* relations within module structures can be challenging. Turn those modules into components, and we can enforce interaction rules at runtime.

Allocation Structures

Expressing the intent behind allocation models in the code used to be extremely difficult. It is not possible to describe and enforce allocation models in the code thanks to the rise of technologies and paradigms such as platform-as-a-service, container technologies (for example, Docker), infrastructure as code, and distributed version control systems.

Treating infrastructure as code creates opportunities for static analysis. Automating build and deployment pipelines to take advantage of cloud-based platforms means we can introduce automated architecture checks into the deployment process. Most platform-as-a-service products can test hardware allocation limits. We can also use configuration and automation to enforce hardware scaling and platform provisioning.

Containers are lightweight and disposable compared to physical hardware and traditional virtual machines. With containers, it is possible to adopt simple and easy-to-enforce allocation patterns such as installing one process per container.

Distributed version control combined with web-based tools such as GitHub makes it easy to allocate teams to specific architectural components while maintaining an open, social development culture. Workflows such as *fork and pull* or *upstream repository* limit access without preventing collaboration.

Add Hints as Comments

Code itself can only take our models so far. We might be able to enforce design decisions, but code constructs won't tell us why those decisions were made.

We can infuse some rationale into the code with good naming and an appropriate use of known patterns. For everything else, there are comments.

Descriptive prose within the code can take many forms. Comments that describe rationale are essential, and you should link liberally to existing design documentation. Even exception messages can contain design hints. We can avoid generic errors by briefly explaining the design rationale behind the error. For example, an *UNKOWN* error is less helpful than *ASSUMPTION_VIOLATED: Document ID required for validation.*

Generate Models from Code

Even when models cannot be seen or enforced in the code, sometimes we can automatically generate models of the system we've built. Depending on the programming language, technologies, and patterns, it may be possible to use models to verify compliance and monitor design evolution automatically.

Any modern object-oriented language can generate UML class and package diagrams. Most programming languages have a dependency analysis tool. Use this generated models to analyze module structures.

Component and connector structures are harder to generate automatically. To generate C&C structures, add instrumentation so that runtime models can be observed. Use the recorded data to generate models and perform other architecture compliance analysis. See Activity 33, *Observe Behavior*, on page 295 for further thoughts on this topic.

Project Lionheart: The Story So Far…

Development has started, though things got off to a rocky start. The team chose to go with conventional tiers pattern for the runtime structures and layers for the code, but there are still many decisions to make. Many team members use different words to describe the same elements in the architecture. As a result, our design discussions end with everyone nodding heads in agreement only to learn later that each person took away a different conclusion.

The code is already a wreck. It reflects the team's lack of understanding of the design decisions made so far. The patterns you thought the team selected for the architecture are nowhere to be found within the system.

After realizing the problems, you call an impromptu whiteboard jam (described on page 255) in an attempt to build consensus around the design decisions and extract a common meta-model. During the whiteboard jam, you encourage teammates to describe the system precisely. Eventually, we settle on some

good names for the concepts, elements, and relations in the models. After the meeting, you formalize the team's conclusions by capturing the meta-model in our wiki.

Now that we have a common meta-model, it's time to refactor the code so that it matches the models we want. Knowing that pair programming is an excellent opportunity to teach architectural principles, you pair with teammates as you fix structures in the code. Luckily it is still early in the system's life, so the refactoring is straightforward.

As you pair with different teammates, you learn that not everyone agrees with or understands the current architecture. Before we get much further along, you decide it would be wise to explore our architecture options in a collaborative workshop.

Next Up

Models help us manage complexity by providing abstractions that we can use for reasoning and communication. There is a conceptual meta-model behind every model. When we know what that meta-model is, we can use it for analysis and communication, and to help us design the architecture.

In this chapter, you learned where models come from and how to describe them, but that doesn't make it any easier to individuate concepts or identify the rules within a meta-model. In the next chapter, you'll learn how to facilitate a group workshop called a design studio that harnesses the power of the group to explore the design space and arrive at good candidate models.

CHAPTER 9

Host an Architecture Design Studio

In nineteenth-century France, architecture professors collected students' projects in wooden carts so they could be easily transported for grading. Of course, nobody was ready for the professor to take their project when the cart came around, but that didn't stop the professor from taking it anyway. As a result, students followed after the cart, feverishly working *en charrette*, in the cart, to finish their bridges and buildings, attaching bits of balsa wood and twists of wire as the professor wheeled the cart off to pick up the next student's project.

In the twenty-first century, *charrette* is a style of workshop inspired by the idea that more time does not necessarily lead to better designs. Architecture professors in nineteenth-century France knew this, and it's a great lesson still today. The user experience (UX) community popularized the charrette as the *design studio*. A design studio encourages group collaboration and has strict time constraints to help the team see a broad range of ideas in a short time frame.

In this chapter, you're going to learn how to plan and facilitate an architecture design studio. We'll start with an overview of the design studio method. From there you'll learn how to facilitate a design studio. Good facilitation, after all, isn't just good manners—it's good business. We'll wrap up the chapter with some tips and hints to ensure your design studio is a huge success.

Plan an Architecture Design Studio

In an architecture design studio, we take advantage of the group's collective wisdom and experience. During a design studio you'll place a strict time constraint on exploration activities so that the group generates as many different ideas as possible, as quickly as they can. We accomplish this by guiding

the group first to explore and then quickly narrow down the field of ideas to a few likely candidates.

A design studio creates buy-in for design decisions and improves team communication by letting everyone have a hand in designing the architecture. We achieve this by keeping the workshop fast, effective, and fun—the *three "Fs"* of design exploration. Fun amplifies engagement, which in turn acts as a force multiplier for the speed and effectiveness of exploration activities.

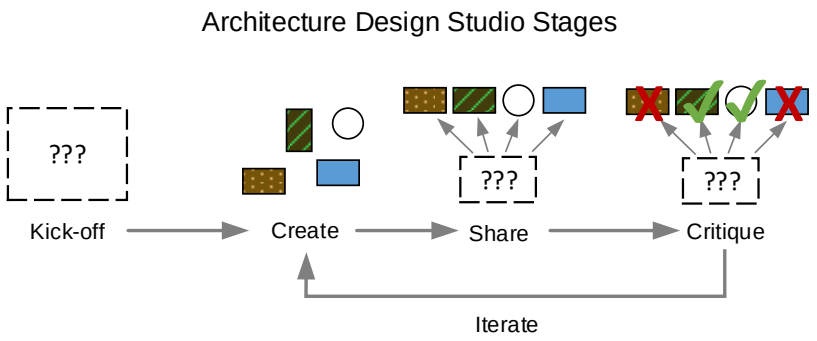
Our job as a design studio host is to plan a workshop that results in a strong set of actionable ideas. Three types of ideas come out of a design studio:

Ideas for things to make. Some ideas will seem promising. The next step for these ideas is to flesh out details by making models or prototypes.

Ideas that need more research. Some ideas will seem right but consist of broad assumptions or miss important information. Plan to investigate these ideas further by running experiments or performing research. Depending on what you learn during the investigation, some ideas might be scratched whereas others become candidates for the architecture.

Ideas that open new questions. Sometimes you might only end up with new questions about the problem we're solving. This is a fantastic outcome for a design studio workshop. It's better to have these questions now than when we've been heads down coding for a few weeks. Take these questions back to stakeholders to improve our understanding of the problem.

A typical workshop lasts anywhere from a few hours up to a day. It's also possible (and in some cases preferable) to let the spirit of your design studio guide your work over the course of several days. No matter how much time you spend, all design studio workshops follow the same basic structure.



1. *Prepare*—Do research to understand the problem you're going to explore.
2. *Kick-off*—Describe the workshop goals and problem context to the group.
3. *Create*—Make models, draw sketches, and build prototypes. Usually a time-boxed activity.
4. *Share*—Present what you've created to the group and describe specifically how your design achieves the goals.
5. *Critique*—The group provides feedback about what you shared relative to how well they think the design satisfies the goals.
6. *Iterate*—Repeat steps 3–5, refining your models and creating new ones. Plan to iterate at least three times for each set of goals you are exploring.
7. *Follow-up*—Decide on next steps for the most promising ideas, risks, or questions.

Let's take a closer look at each stage of the workshop.

Before the Workshop: Prepare

Before kicking off a design studio workshop, we'll need to choose goals and decide who to invite (see *Invite the Right Participants*, on page 120). Gathering a group to design architecture is not a good use of anyone's time until you at least partially understand the problem you'll to explore.

Preparing for a workshop can take serious time and should not be underestimated. Talk to stakeholders. Do your research. Work to define the business goals. Refine quality attributes and other architecturally significant requirements (ASRs). Before starting the workshop, you should understand enough of the problem and context so that you can articulate useful workshop goals.

Define one or two workshop goals that clearly describe what the group will explore during the workshop. You don't want one person designing a database while someone else is working on deadlock problems. It's OK if you only partially understand the problem before starting the workshop. Exploring solutions is one way to gain a deeper understanding of the problem.

Workshop goals tell participants how they will spend their brainpower during the workshop. There are a few examples in the table on page 116.

Preparing for the workshop might take a few days or even weeks. You should at least have draft business goals and architecturally significant requirements before starting. Once you have a handle on the project context and a reasonable workshop goal, you're ready to run the workshop.

If your goal is to explore…	You might say…
A specific set of quality attributes	"Scalability and reliability are our top two quality attributes. How can we promote these at the same time?" (You'll share the specific scenarios with everyone.)
Interfaces between components	"We've decided to use REST for our APIs. Let's figure out what that really means for our system." (Assumes everyone knows what is involved with RESTful architecture)
Domain models	"Based on our stakeholders' business, we need to define some common abstractions that we'll use throughout the system."
How to get out of a jam	"We're seeing a size and scale of data we didn't anticipate ten years ago when we first designed the system. Let's figure out as many options as we can."
Allocation structures	"We need to partition the system so we maximize parallel development effort."
Pattern selection	"We need to decide how we're going to get data from point A to point B. Before we go into pros and cons, let's understand the options."
Many different ideas	"Today we want to see as many different ideas as possible. Everyone should try for at least 5 ideas."

Kick Off the Workshop

Start the workshop by setting the stage for a successful collaboration. Make sure everyone has the proper context by sharing what you know so far about the problem and the architecture. Review anything relevant to the area you plan to explore. The amount of workshop time devoted to sharing context should be commensurate with the group's background knowledge. If this is the first time some people have seen business goals or ASRs, spend more time describing the context.

After reviewing the context, outline the workshop goals. These goals are used throughout the workshop and keep the group focused. During the workshop, everyone will create designs to satisfy the goals. We'll also critique design ideas with these goals in mind.

Iterate through the Create-Share-Critique Cycle

We'll spend most of the workshop in the create-share-critique cycle.[1,2] Each iteration through the cycle explores more of the design space and improves the group's understanding of what is possible. We'll talk about specific activities we can plug into this cycle in *Choose Appropriate Design Activities*, on page 119.

Create

During the create step, participants work alone or in small groups to address workshop goals by sketching and modeling their design ideas. Keeping it analog works best during a workshop. Using pens or fat markers and paper encourages people to focus on ideas rather than precision. At this stage of the game, you value ideas, not perfection.

The create step is always time boxed. Short workshops might spend only 5–7 minutes in the create step. Longer workshops can allow for more time. Keep in mind that more time does not always lead to more or better ideas! Architecture exploration requires deep thought, so adjust time relative to your goals. Anything less than 5 minutes is not enough time for most software architecture topics.

Share

After everyone has created something, it's time to share it with the larger group. This step is sometimes called *pitch*, as in *make a pitch* for your idea. Give each group 3–5 minutes to share what they created and describe how their design satisfies the goal. Groups should hit only the main points and avoid giving a full briefing. Participants will learn quickly not to create more than they can share.

During the share step, workshop participants listen and may not ask questions. Everyone will have a chance to ask questions and share comments during the critique step.

Critique

Immediately after a group shares their design, give the other participants an opportunity to critique the ideas. Feedback during the critique should focus on the merits of the design relative to the workshop's goals. The idea is to lay the foundation for a constructive dialogue. How does the design fall short of satisfying a goal? Why does the design not meet the specified need?

1. https://vimeo.com/37861987
2. http://www.madpow.com/~/media/files/designstudio-webinar.ashx

During the critiques, encourage everyone to be specific and focus on facts.

Do this	Avoid this
Do: Focus on the goals the designers said they addressed.	Don't: Get defensive about your design.
Do: Be specific; focus on facts.	Don't: Share personal opinions—"I like XYZ, it's my favorite."
Do: Ask clarifying questions.	Don't: Get sidetracked by problem solving.
Do: Point out risks and new problems introduced by the design.	Don't: Be a jerk (your turn is next!).
Do: Point out benefits about the design.	Don't: Focus only on downsides.

Critiques should point out good things about a design as well as things that can be improved. Even terrible designs have a few good ideas, and there is always room for improvement even in the best designs. Every design should receive a mix of positive feedback and constructive criticism. Once all groups have shared their ideas, do it all over again.

Iterate

The purpose of the create-share-critique cycle is to promote rapid divergence and convergence of thinking as described in *Diverge to See Options, Converge to Decide*, on page 63. During the create step, we generate new ideas. During the share step, we create opportunities for cross-pollination and serendipitous inspiration. During the critique step, we eliminate dead ends and nudge people toward better (or at least different) solutions.

Iteration allows participants to build on what they collectively learn. Move through the create, share, and critique steps as quickly as you can while remaining effective.

With each iteration, tweak the group dynamics. Tweaking group dynamics between iterations encourages broader exploration while simultaneously building a sense of shared ownership. If participants are working alone, have them work in pairs or small groups. If they are working in groups, try mixing up the groups. Plan for at least three iterations of the full create-share-critique cycle for each set of goals in the workshop.

Close the Design Studio and Decide on Follow-up Actions

A strong finish can mean all the difference between a productive workshop and a fun waste of time. Allow time at the end of the workshop to reflect on emergent themes and discuss general observations as a group. Also, decide

on specific action items. Which ideas seem promising and should receive more attention? Were any significant risks raised that should be addressed? Are there any experiments that should be started?

Take pictures of all the material produced during the workshop. Create write-ups in a shared repository while the ideas are still fresh in everyone's minds. Most importantly, record the action items and follow up with individuals to make sure they take the next steps.

Choose Appropriate Design Activities

As the design studio host, we're responsible for choosing activities that guide everyone through the create-share-critique steps in a fast, effective, and fun way. There are many design activities we could use, though not all design activities are appropriate for architecture design. Try to select activities that are architecturally focused and effective when thinking about the system as a whole.

Here is an example workshop agenda based on the round-robin design activity. The workshop itself might run anywhere from 90 minutes if we're only exploring initial ideas to a whole day if we're well prepared and plan to explore multiple parts of the system deeply.

Activity	Timing	Purpose
Introduce the context and goals	15 minutes	Arm everyone with the knowledge they'll need to be an active, contributing participant in the workshop.
Round-robin design activity, described on page 252	30 minutes	Promotes rapid divergence and convergence to get the workshop rolling. This agenda plans for only one round, but you could do more with more time.
Group Poster Activity, described on page 249	30 minutes	Summarize findings as posters so they can be shared more easily among the group. Start building consensus.
Present and critique posters	15 minutes	Allow about 3 minutes per poster for presentations. Use dot voting for the critiques.
Reflect and review action items	10 minutes	Review how the workshop went and define next steps to ensure strong follow-up. Allow about 10% of the workshop time for this.

To customize the workshop, use different exploration activities. Instead of a round-robin design, try a whiteboard jam, described on page 255. If your aim is to arrive at a better system metaphor, try telling stories by pretending the architecture has human qualities, outlined on page 226. The activities you choose for the design studio should help you achieve your goals.

 Get Your Hands Dirty: Sketching Practice

During a design studio, we ask participants to sketch quickly and share big ideas under extreme time pressure. Sketching and sharing can be difficult if you haven't practiced. Practice sketching architectures so you are well prepared to help participants who freeze up during a workshop.

Here are some things to think about:

- How many different ways can you draw a line? An arrow? What meaning might the different lines and arrows convey?
- Try to draw the most precise architecture diagram you can. Now draw the same views and see how much precision you can remove without creating ambiguity.
- Draw as many different types of people as you can. Find a style that works for you.
- Try filling a whole page with shapes, arrows, people, and doodles.
- For real practice, purchase a pocket notebook and fill it with sketches.

Invite the Right Participants

With any group exercise, the quality of the workshop is determined by the participants. Too many people in a workshop makes for an expensive meeting. The wrong people in a workshop can limit how wide you explore and might even bump constructive discussions off course. As the host, you need to balance two variables: size and diversity.

Right-Size the Workshop

Effective collaboration breaks down in groups larger than about seven people. Large groups are difficult to manage, require more time for communication, and are impossible to coordinate for scheduling. One way to manage a large group is by dividing it into smaller ones. Even then, a single facilitator can realistically handle only three or four subgroups at a time.

Depending on what you want to achieve and whether you have a co-facilitator available, limit the size of an architecture design studio to about ten people. As a rule of thumb, work with the smallest group that can still effectively explore. If you need to pair with only one other person to work through an idea, then work with just that one person. A group of about 3–5 people seems to be a good sweet spot for many software architecture design tasks.

Invite a Diverse Audience

The ideal architecture design studio always includes at least one person who can offer a dissenting opinion or who brings a different perspective. Including someone with a different background or a fresh perspective creates greater opportunities for eureka moments.

Start by inviting essential stakeholders. Also include someone who knows little about your particular problem and can attack it from a different perspective. If your team is programmer heavy, invite a tester or product manager. If you are all systems developers, invite someone knowledgeable in front-end development. Bring in people who are good at asking questions or thinking about complex ideas. Ensure you have a range of experience across a variety of topic areas.

Everyone has a unique perspective. These differences can help the group explore further and wider than if everyone thought the same about the design.

Harness the Power of Groups Wisely

While all design is social, this does not mean all design must be done in groups. Group collaboration does have a potential dark side. *Groupthink* is a phenomenon where the group loses its individuality and worries more about harmony and consensus than satisfying the goals of the workshop. When a workshop falls into groupthink, the decisions they make will be suboptimal and sometimes even be harmful.

Seasoned basketball coaches can tell how well their team is doing by listening to the squeaks players' shoes make on the court. Likewise, a healthy design studio workshop has an unmistakable *hummm* of collaborative brainstorming. There are some specific things to listen for to determine if your group is collaborating well in the table on page 122.

Proactive facilitation is our best tool for effectively harnessing the power of the group. Look out for the silent majority who seem to just go with the flow. Discussions lacking in disagreement may seem like positive progress but is

If the group…	It might mean…
Asks questions to clarify meaning, politely challenges ideas, and discusses implications of an idea	Everyone is collaborating well
Goes along with whatever seems to be the prevailing idea; hesitates to share their thoughts	Potential fear of conflict or lack of confidence in collaborating
Does not share a wide range of ideas; acts as an echo chamber; comes back to the same themes	The group didn't diverge their thinking widely enough
Always lets the same people do the talking	Not everyone understands the discussion; dominant personalities are overwhelming quieter individuals

more likely to be the opposite. Conflict defines the boundaries of exploration and highlights important concepts.

Manage the Group

Facilitation is more than just making an agenda and keeping time. Facilitation is an active role. How you share an activity has significant influence over how participants approach it. How you interact with participants can alter their behavior in the workshop. It's the facilitator's responsibility to keep the design studio moving and ensure the workshop produces useful outcomes.

Allow Enough Time for the Workshop

Running out of time or rushing through activities can undermine the workshop goals and decrease participants' confidence in the findings. You want to see a broad range of ideas in a short amount of time so that you can cheat bounded rationality (see *Find a Design That Satisfices*, on page 27).

It might be possible to complete a rapid exploration workshop with only one or two goals in an hour or two. Such workshops are ideal for exploring a small number of narrow goals or for building consensus when the group understands the high-level solution but needs to explore details.

A design studio with many goals might need one or two full days to complete. When the problem is not well understood, plan on having many smaller sessions over the course of several weeks. Remember that not every problem can be explored collaboratively in a workshop setting.

Set Expectations from the Start

Great workshops have some degree of mystery but also let participants know up front what they'll be doing and why you're here. State the workshop goals up front and ensure the group is on board before starting.

Start a workshop by sharing the general workshop agenda. It's not necessary to share every detail. Some details we'll want to keep secret to prevent participants from getting confused or "pre-fetching" designs. For workshops running more than a few hours, share estimated start times for agenda items so participants can self-select in or out of specific activities. This way participants will be present at the workshop and deal with distractions at times that won't disrupt the workshop.

Set ground rules at the start of the workshop. Here are some examples of ground rules:

Sample Workshop Ground Rules	
Everyone participates	When time is up, we move on
No "right" or "wrong" answers	Ask questions if you need help
Watch the clock (I'll help too)	Have fun (seriously) ☺

Introduce Activities with the Tell-Show-Tell Approach

When introducing a new activity to the group, always tell participants what they'll do, show them an example of what it looks like, then review the instructions you just gave them. Most people will miss important details the first time they see something new. Reviewing instructions after seeing a concrete example gives participants a second chance to ask questions about the activity.

It's best to use examples from previous workshops. When examples don't exist, create a mock-up by staging a picture of the activity or approximating an example.

Share Tips for Activities

Inevitably when you say *Go!*, someone will freeze up. For many participants, this workshop could be their first time working collaboratively like this.

To help participants get started, share tips for each activity. A simple reminder or nugget of advice is an excellent way to help participants avoid blank page syndrome. Keep an eye out for groups or individuals staring at a blank page—they may need help getting started.

Set Deadlines

All activities in the architecture design studio are time boxed. Set realistic but aggressive time constraints to keep things moving. Participants should feel rushed but somehow manage to finish the activities just in time. Ideally, every activity is a buzzer beater with groups putting finishing touches on their sketches as you call *Time's up!*

Educate Participants Just-in-Time

Unless you've collaborated with all the participants in your workshop before, set aside some time to teach people critical software architecture and design concepts just-in-time during the workshop. The goal with just-in-time education is to ensure participants have just enough information to be successful in the particular design activity you're doing. A quick review of important architectural concepts or an introduction to quality attribute scenarios might be all that's required.

Participation is essential to a workshop's success. It's why you gathered a group in the first place! Ensure participants have the knowledge they need to participate effectively.

Use Parking Lots

Design is a journey that often takes a winding road to reach a destination. During a design studio, you may happen upon interesting ideas and useful discussions that you don't have time to explore at that exact moment. Keep a running list, a parking lot, of discussion points to be visited at the end of the workshop. Using parking lots keeps the workshop moving and assures interested parties there will be time to discuss topics that interest them.

Work with Remote Teams

It's not always practical to gather a group of people for a design workshop. Luckily, design studios and other collaboration-focused workshops work great for remote teams. Here are a few tips for facilitating a remote workshop. These tips apply to any of the activities discussed here as well as the activities described in Part III.

Use remote collaboration tools. This is an absolute prerequisite. Find a combination of remote collaboration tools that allow your group to work together and share the fruits of your exploration. Depending on the specific activities in the workshop, you will need tools that allow screen sharing, collaborative document editing, collaborative drawing, brainstorming, group

chat, and voice communication. Many tools are available on the web for each of these options.

Add time to the agenda. Distributed groups need more time to complete collaborative design activities. Remote meetings always experience technical problems. Plan ahead and you won't be surprised.

Create breakout opportunities. Only one person can talk on the phone at a ftime. When only one person can speak at a time, group work becomes nearly impossible. Create a back channel for communication using group chat software. If group work is part of the workshop, decide which teleconference phone numbers they'll use and set clear deadlines for when the large group should reconvene.

Provide a focal point. It's easy to get distracted in a remote meeting. Facilitators don't have the ability to sketch notes on a whiteboard—only the people in the room with you will see it. Prepare presentation material that participants can use on their own or share your screen. Invite everyone to contribute to a shared document during group discussions to keep everyone engaged.

Make it face-to-face. Nothing beats face-to-face interaction. When possible, use video conferencing software that allows participants to see one another for at least parts of the workshop.

Take it off line. Workshops aren't the only way to explore ideas. You can run many design activities in *slow motion*, over the course of several days. For example, a round-robin-like activity can be accomplished via email just as effectively as in person.

Here's an example of what a remote architecture design studio looks like. Marie, our facilitator, arrived early, started her screen sharing software, and dialed into the meeting's teleconference number. Once all participants were dialed in, Marie kicked off the meeting by presenting slides containing the workshop's agenda and goals. Marie would usually write these things on a whiteboard, but she wanted all participants to be able to see them.

The team was doing a round-robin design activity. Marie instructed participants to sketch a view of the architecture and take a picture with their phone to share it. Each participant shared their sketch via a Slack private message. For the second round, participants annotated the picture they were assigned using drawing software, took a screen shot of it, and added the original and annotated images to a shared Box.com folder.

After some brief discussion about the sketches, Marie divided participants into groups. Each group used Google Hangouts and shared Box.com documents to create a presentation with their ideas for the architecture. At the requested time, everyone rejoined the workshop teleconference to share their presentations. During the critiques, Marie and other participants took notes together in a shared document. Instead of a rapid-fire 90-minute workshop, the design studio ended almost on time after a little more than two hours.

Project Lionheart: The Story So Far…

The team is at a crossroad. Some team members feel we should use a service-oriented approach using microservices. Others feel we should play it safe and stick with a tried-and-true multi-tier pattern. You need to resolve the conflict and create buy-in for the design decision. Since either pattern would probably work out fine, you host a design studio to help the team decide.

Your goal for the workshop is to explore the nuances of each pattern and flush risks into the open for the team to discuss. You start the workshop with small-group exercises to generate ideas. The microservices and multi-tiered patterns come up, but a few other interesting and unexpected ideas are raised as well.

After the initial round of presentations and critiques, you have us create group posters. Unexpectedly, as the team works through the different ideas, microservices fall completely out of favor! By the end of the two-hour workshop, the team explored a half dozen design options and arrive at a great solution. More importantly, everyone had a say in the final decision and there seems to be a genuine sense of shared ownership among the team that didn't exist before the workshop.

Next Up

A design studio is a fantastic method for quickly exploring ideas. Perhaps even more important is the journey your team takes to arrive at a decision. Everyone who participates has a stake in the design they helped create. A sense of shared ownership encourages greater autonomy and a sense of responsibility. This shared ownership permeates all aspects of design from code to architecture.

Collaborative design workshops are a powerful tool, but as we've seen so far, there is more to architecture design than group work and sticky notes. A design studio alone is not sufficient for designing an amazing architecture.

Its effectiveness depends greatly on the context in which it's used. We still need to think and do the work.

So far you've learned how to define the problem in a way that aides architecture design, how to explore design concepts, how to create models, and how to make design decisions. In the next chapter, you'll learn how to visualize design ideas so you can share them with your stakeholders and team.

CHAPTER 10

Visualize Design Decisions

The best way to share an idea is to make it tangible. Tell me about your architecture and I may not understand you. Show me your architecture, and I can explore it at my own pace, using my preferred cognitive style. Developers intuitively know this. Need to talk about an abstract idea? Find a whiteboard and start sketching. If we can draw the idea we're trying to share, then we'll know our imaginations are in sync.

Anyone can draw. Architecture diagrams don't need to be pretty, but they do need to share ideas effectively. In Chapter 8, *Manage Complexity*, on page 99 you learned how to create accurate models so you can reason about how well the architecture promotes desired quality attributes. In this chapter, you will learn how to draw architecture diagrams to enhance communication among developers.

Show the Architecture from Different Views

It's impossible to capture every interesting detail about the architecture in a single picture. Instead of trying to put everything into a single diagram, we'll create multiple views of the architecture. A *view* is a story about the architecture told from the perspective of a particular stakeholder or set of related concerns.

Think about how we use a mapping application like Google Maps to plan a road trip. Zoom in and we can see city streets. Zoom out and whole cities become just dots along interstate highways. Apply an overlay and we can see real-time traffic or weather. We can even swap maps of streets for satellite images or topography. Some applications offer street-level perspectives, which allow us to see the world at eye level from a snapshot in time.

We can use these different views of the world in our mapping application to plan a road trip. What's the best route from Pittsburgh to Albuquerque? How

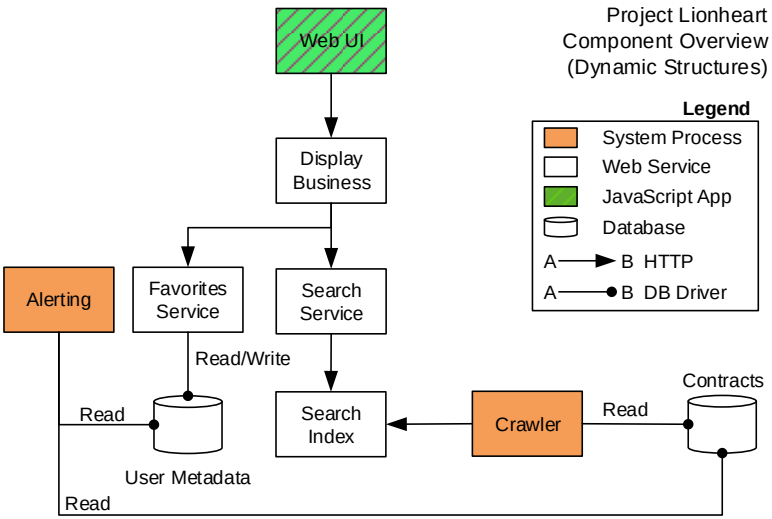
do we avoid that thunderstorm? Where is a good place to eat chicken chow mien along our route?

Just like a map, each view of the architecture helps stakeholders answer questions about the system. How is development progressing? Who is working on which components? How will we support our quality attribute scenarios?

There isn't a fixed set of views for every software system, but there are several views that are useful for most software systems. Let's explore some of these useful views in more detail.

Tell Us What Elements Do with an Element-Responsibility View

Lines and boxes are the architect's most used tool and her least trustworthy companion. Box and line diagrams make complex relationships among elements easy to see, but their architectural meaning is never self-evident. Recall this overview diagram of the Project Lionheart services:



This diagram is missing vital information. We know every element serves a specific purpose, but a picture alone is not a view. The element-responsibility catalog shown on page 73 for this diagram fills in the missing pieces. Depending on the information we need to share, we can express responsibilities as diagram annotations, in a table, or as descriptive prose.

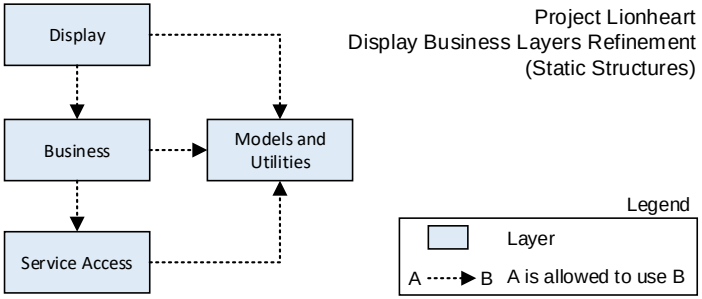
Element-responsibility views are extremely common. In some cases, this might be the only view you need. If you can list the elements and their responsibilities, then you stand a chance of building a working system.

Zoom In or Out with a Refinement View

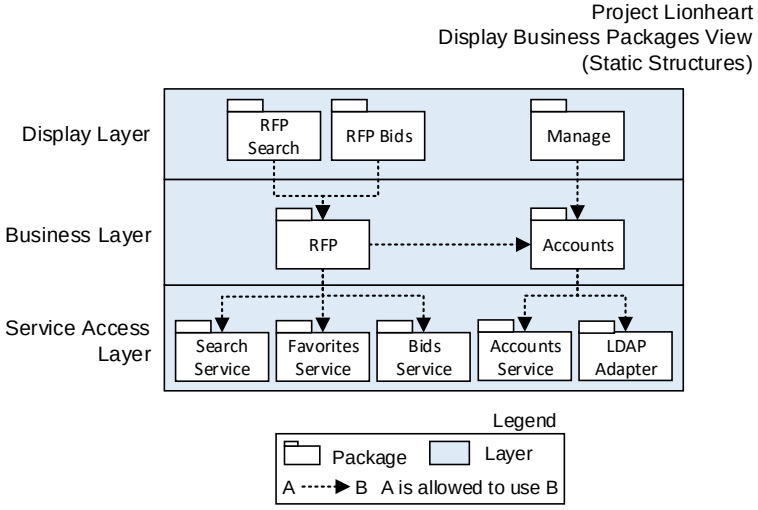
I always love that scene in a television crime show when the hero cracks the case by enhancing a blurry image. *I can't see his face. Zoom in. Enhance!* The cool thing about software, unlike blurry images, is that we actually can zoom in and enhance nearly infinitely.

Refinement is the process of increasing detail in a model over a series of views. It's like zooming in on a structure to show the elements' inner workings, enhancing, and then zooming in again.

Let's zoom into the Lionheart Display Business component to take a closer look at some of its static structures.



At one level of refinement, we can see the Display Business component uses a typical layered pattern: one layer for display, business, and service access logic, with a sidecar layer to organize data models and utility classes. This diagram establishes context, but we might learn more about maintainability if we refine further.



In this refinement, we show packages inside the layers and how they interact with one another so we can reason about maintainability quality attribute scenarios. Looking at this refinement, it's clear that the *RFP Package* might be too interconnected, which makes testing and debugging difficult.

Notice that the *Models and Utilities Layer* is not shown in this refinement. Everything is *allowed to use* classes in the *Models and Utilities Layer*. Including all the relations would make the diagram too cluttered for analysis, so we eliminated some relations from this view. Slicing and dicing views is useful but can make the architecture models harder to understand.

Refinement views help us focus on the details needed to answer specific questions about the architecture. Coarse-grained refinements provide a big picture view of the world. After we've established the context, we can show finer-grained refinements by zooming in to show important details needed by specific stakeholders.

Use the principles of architecture minimalism described in *Preserve Ambiguity*, on page 16 to decide when to stop refining a model. Only refine to a level of detail necessary to demonstrate specific quality attributes and reduce high-priority risks.

Show How the Architecture Promotes Quality Attributes

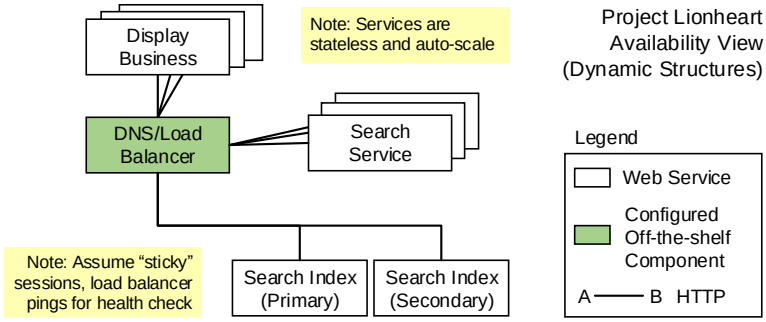
A *quality attribute view* demonstrates how the architecture achieves specific quality attributes. Quality attribute views might hide details not relevant to the current discussion, or highlight details relevant only to the given quality attribute. For example, consider this availability scenario for Project Lionheart from the table on page 54.

> A user searches for open RFPs and receives a list of available RFPs 99% of the time on average over the course of the year.

To satisfy this quality attribute, we introduced a redundancy pattern. Let's create a view as shown in the figure on page 133.

Promoting availability means our Lionheart services must be resilient in the face of failures. To accomplish this, we'll need multiple instances of the *Display Business*, *Search Service*, and *Search Index* components. Since the *Display Business* and *Search Service* components are stateless microservices, we can easily deploy multiple instances in any container management system such as Kubernetes[1] or Marathon.[2]

1. http://kubernetes.io/
2. https://mesosphere.github.io/marathon/



The *Search Index* is stateful and a potential performance bottleneck for our system, so we'll need to be more careful about data storage. We'll also need to think through routing when there is a fault to avoid downtime and data partitioning. To keep things simple, we'll use a load balancer and Domain Name System (DNS) to route requests.

Let's use the diagram to determine whether we satisfied the quality attribute scenario. Pretend one of the *Search Index* components fail. The load balancer detects the failure with a health check and routes requests to the secondary *Search Index*. So far, so good.

This view is moving in a good direction but needs more work. How often does the health check occur? What are the requirements for a ping? What happens if the load balancer goes down? What happens when a failed *Search Index* comes back online? These questions and more should be described in explanatory prose accompanying this diagram. One diagram might not be enough to thoroughly explain how the architecture satisfies our availability scenario.

Connect Elements from Different Views

With many views in play, it's useful to see how elements in different views are related to one another. A *mapping view* serves just this purpose by combining two or more views into a new view that shows how the elements are related.

Two useful mapping views are *work assignment* and *deployment*. In a work assignment view, we show who works on different parts of the system by mapping teams with the elements they'll build. Deployment views show where runtime elements from a component and connector view will be installed and used.

Here's an example of a work assignment view for Project Lionheart:

Component	Team Assigned	Notes
Web UI	Honey Badgers	Team consists of front-end web development pros
Display Business	Honey Badgers	Component is tightly coupled with Web UI
Search Service	Red Shirts	
Search Index	Red Shirts	Team of experienced Solr developers
Favorites Service	Honey Badgers	Team has capacity; component directly impacts user experience
Alerting	Open/Unstaffed	First team available
Crawler	Red Shirts	Team has expertise
DNS/Load Balancer	Tron	Infrastructure experts
User metadata and contracts databases	City of Springfield	They own the databases

Notice that this view is not a diagram. Our objective is to communicate design decisions by any means necessary. Sometimes a simple table is all that's needed to get the job done.

Mappings create connections between stakeholders with different concerns. This work assignment view is perfect for a project manager who needs to create a schedule or staffing plan.

Mapping views provide a layer of context that can be difficult for stakeholders to piece together on their own. Consider a product manager's needs. Maybe they want to know when certain features will be ready to ship. A mapping between architectural components and value-adding features would help everyone understand which parts of the software support different features. This tiny smidge of knowledge enables team self-organization and helps developers prioritize work with the product manager's needs in mind. Now you're empathizing with stakeholders!

Let Ideas Breathe with a Cartoon

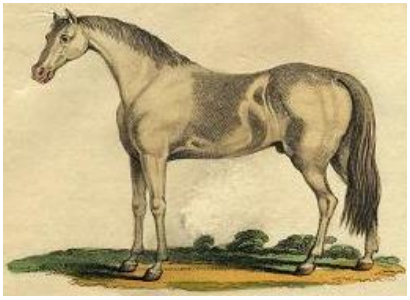
All the views we've seen so far have been rather precise. Precision has a cost. Precise models require exacting details and are time-consuming to create. Sometimes precision gets in the way of communication, especially when you're

exploring ideas. As we're getting to know the system, it's sometimes useful to create accurate models with less precision.

An *architecture cartoon* is a fast-to-create, imprecise model that favors communication over analysis. Use cartoons for rapid iteration and informal communication. They are perfect for exploration workshops and impromptu discussions.

Cartoons capture the essence of the idea you want to share. Here is a cartoon of a horse and an anatomically precise diagram of a horse. A horse has four legs, a bushy tail, and a mane. A more precise drawing might help if we need details about the musculature or anatomy. But if we just need a placeholder for a horse, then my cartoon works fine.

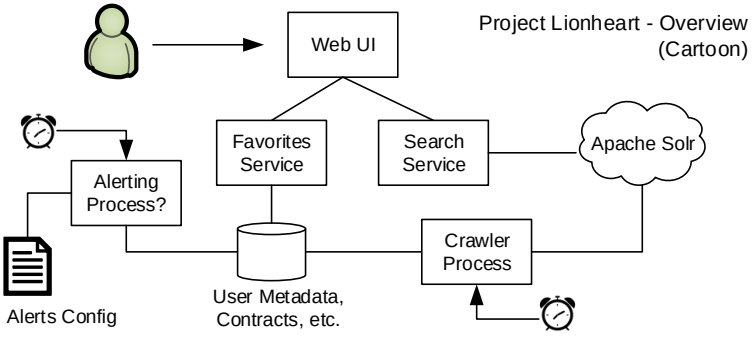
A Horse! drawn by M. Keeling,
Sharpie on recycled printer paper, 2017



From 1. Arabian Horse. 2. Zebra. 3. Ass engraved by T. Dixon, published in An History of the Earth and Animated Nature by Oliver Goldsmith, 1822

Architecture cartoons use simple notations and ignore many of the best practices we'll discuss throughout this chapter. Using loose notations is OK, especially while you're working through an idea. Once design decisions start to converge, then it's time to create a more precise model.

Bringing this back to architecture, here's a cartoon of the Project Lionheart Overview. Compare it with the more precise views we've seen so far.



Create Custom Views to Show Exactly What You Need

Any view of the architecture that helps you effectively tell a story about the system to stakeholders is a view worth having. Get creative and make custom views for your particular purpose.

Views always combine multiple variables. Elements and responsibilities. Quality attributes and patterns. Elements and project schedules. Inventing a new view can be as simple as combining new variables with architectural elements. Need to show performance bottlenecks in the system? Start with a component diagram that shows information flow, color-code components based on execution time, and voilà, we've created a performance view.

Remember, all views, even custom views, are governed by an underlying meta-model as discussed in *Design the Meta-Model*, on page 101.

Draw Fantastic Diagrams

Great diagrams are not just beautiful pictures. Great diagrams are accurate models that reflect the conceptual underpinnings of the architecture. Architects may be infamous for drawing box and line diagrams, but there's more going on in these diagrams than just coarse-grained abstractions.

As you learned in *Show the Architecture from Different Views*, on page 129, you can show many different design ideas with diagrams. Visualizing the architecture with a diagram makes it tangible in a way that other mediums cannot. Fantastic diagrams make the architecture accessible to everyone.

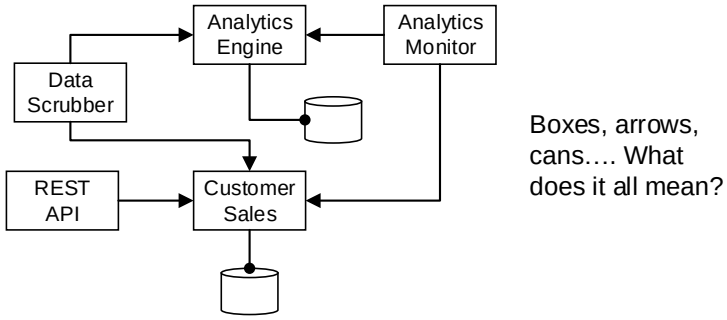
Here are some tips for creating fantastic diagrams:

Do this	Avoid this
Do: Create a legend that summarizes parts of the meta-model relevant to this diagram.	Don't: Assume your readers know your notations (even with the UML).
Do: Add a descriptive title and tell what kinds of structures are in the diagram.	Don't: Try to include everything in a single diagram.
Do: Add text annotations to enhance clarity.	Don't: Use notations that lose meaning when printed in black and white.
Do: Use a consistent notation across all diagrams.	Don't: Go overboard with superfluous flourishes or an excessive variety of shapes and lines.
Do: Make the patterns visible.	Don't: Skip descriptive prose.

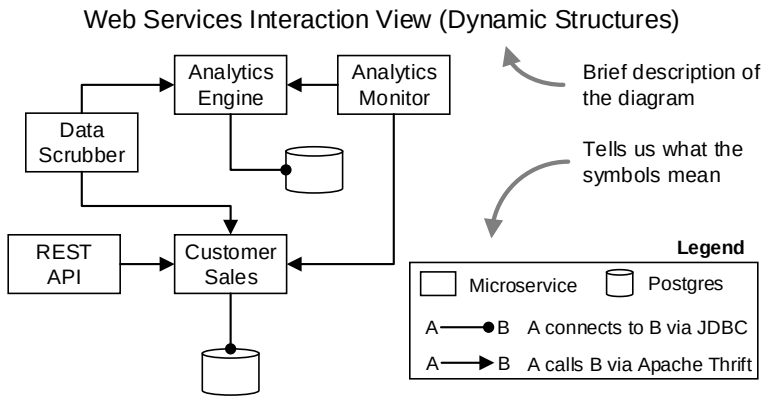
Let's explore these tips in more detail.

Use the Legend, Don't Just Include a Legend

What do the symbols in this diagram represent?



Without a legend, it is impossible for us to know what's happening. Here is the same diagram, this time with a legend:



We can see immediately that we're dealing with fine-grained web services and that the communication mechanism relies on Apache Thrift.[3] Downstream designers responsible for implementing a component in this architecture will want to know these details. Other stakeholders can use this information as the basis for further conversations.

The legend introduces the architecture's conceptual meta-model (introduced on page 101) to our audience. Draw the legend first and use it to keep our diagrams consistent with reality and turn them into tools for analysis.

Now that we have a common understanding of the meta-model, we start to notice mistakes and have questions about the diagram. Should the *Analytics*

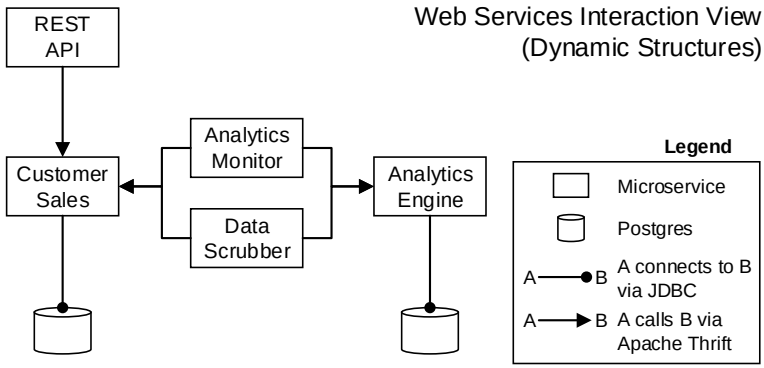
3. http://thrift.apache.org/

Monitor and *Data Scrubber* be microservices? What if I told you the *Analytics Engine* service provided a REST interface and not Thrift? Is this a mistake, gap in knowledge, or plan for the future?

No matter what notation you use, every diagram should have a legend. This advice applies to standardized notations such as the UML as well as custom notations. Not everyone who sees our diagram will be familiar with the dialect of UML you use. Legends enhance meaning.

Highlight the Patterns

The previous diagram is hiding a secret. Look what happens if we move some of the components around. Notice that we only changed the elements' positions.



Rearranging the services reveals that services in this architecture are allocated using a multi-tier pattern! This rearrangement might seem inconsequential, but making the patterns visible communicates the intent behind the architecture to downstream designers. For example, knowing this is a multi-tier pattern, I would not expect the *REST API* to communicate directly with a database from the *Data Tier*.